locations, computing improvement-based quantities can still be done easily under a Gaussian process. By the end of the chapter, we will gain a thorough understanding of what a BayesOpt policy needs to do, and how this is done with the two improvement-based policies. We will also learn how to integrate BoTorch, the BayesOpt library in Python that we will use in this chapter through to the end of the book, to implement BayesOpt policies.

4.1 Navigating the search space in Bayesian optimization

How can we make sure that we are using our past data to inform future decisions correctly? What are we looking for in a BayesOpt policy as an automated decision-making procedure? This section will answer these questions and give us a clear idea of how BayesOpt works when using a Gaussian process (GP).

Specifically, in this next subsection, we reinspect the BayesOpt loop that was briefly introduced in Section 1.2.2 of Chapter 1 to see how decision making via BayesOpt policies is done in tandem with training a GP. Then, we discuss the main challenge that BayesOpt policies need to address, which is the balance between exploring regions with high uncertainty and exploiting good regions in the search space.

Take as an example Figure 4.1, which shows a GP trained on two data points at *1* and *2*. Here, a BayesOpt policy needs to decide at which point between *-5* and *5* we should evaluate the objective function next. The exploration–exploitation tradeoff is clear: should we inspect the objective function at the left and right extremes of our search space (around -5 and 5), where there's considerable uncertainty in our predictions, or should we stay within the region around 0, where the mean prediction is at the highest?

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>